Alexandria University

Faculty of Engineering
Computer and Communication Engineering

Digital Communications

Final Project:

Performance of Different Modulation Techniques

Group Members:

Ahmad Yasser Mohammad Salaheldin Hamad (ID: 9074)

Ahmed Walled Aref Mahmoud (ID: 9180)

Omar Adel Abdallah Masry (ID: 8746)

Contents

1 Part I: Comparison of OOK, PRK, and BFSK **2**
1.1 Introduction 2
1.2 Theoretical Background 2
1.2.1 Signal-Space Representation 2
1.2.2 Probability of Error (Coherent Detection) 2
1.3 Simulation Methodology 3
1.4 Simulation Results 3
1.5 Conclusion (Part I) 4

2 Part II: Comparison of M-ary ASK ($M = 2, 4, 8$) **5**
2.1 Simulation Results 5
2.2 Discussion 5

3 Part III: Comparison of Rectangular BPSK, QPSK, 4-QAM, and 4-ASK **7**
3.1 Simulation Results 7
3.2 Discussion 7

Appendix: Complete MATLAB Simulation Code **9**

1 Part I: Comparison of OOK, PRK, and BFSK

1.1 Introduction

This section compares the bit-error-rate (BER) performance of three fundamental binary modulation schemes — On-Off Keying (OOK), Phase-Reversal Keying (PRK), and Binary Frequency-Shift Keying (BFSK) — over an additive white Gaussian noise (AWGN) channel. Each scheme is examined first through its signal-space representation, then validated by Monte-Carlo simulation over 10^6 bits per SNR point across the range 0–60 dB in 3 dB steps.

1.2 Theoretical Background

1.2.1 Signal-Space Representation

Each modulation scheme maps a binary bit to a point in a signal-space whose geometry directly governs the error probability. The optimum coherent receiver projects the received signal onto the basis functions and applies a minimum-distance decision rule.

- **OOK** — A one-dimensional, unipolar constellation. Bit $1 \to +\sqrt{E_b}$, Bit $0 \to 0$. The two symbols are separated by a distance $d = \sqrt{E_b}$, and only half the symbols carry energy, so the average bit energy is $E_b/2$.
- **PRK** — A one-dimensional, antipodal constellation (a.k.a. BPSK). Bit $1 \to +\sqrt{E_b}$, Bit $0 \to -\sqrt{E_b}$. The symbols are separated by $d = 2\sqrt{E_b}$ — the maximum achievable distance for unit-energy 1-D signaling.
- **BFSK** — A two-dimensional, orthogonal constellation. Bit $0 \to (\sqrt{E_b}, 0)$, Bit $1 \to (0, \sqrt{E_b})$. The two symbols are separated by $d = \sqrt{2E_b}$, placing BFSK between OOK and PRK in terms of geometric robustness.

1.2.2 Probability of Error (Coherent Detection)

Using the Q-function $Q(x) = \frac{1}{2}\text{erfc}(x/\sqrt{2})$, the closed-form bit-error probabilities are:

$$\text{OOK: } P_e = Q\left(\sqrt{\frac{E_b}{N_0}}\right)$$

$$\text{PRK: } P_e = Q\left(\sqrt{\frac{2E_b}{N_0}}\right)$$

$$\text{BFSK: } P_e = Q\left(\sqrt{\frac{E_b}{N_0}}\right)$$

PRK enjoys a 3 dB advantage over both OOK and BFSK because of its antipodal geometry — doubling the squared distance between symbols halves the argument of the Q-function in dB terms. OOK and BFSK share the same expression, so their curves should overlap when compared on an equal-average-energy basis.

1.3 Simulation Methodology

A common pseudo-random bit stream of length $N = 10^6$ is generated once and reused across all three modulators, ensuring a fair comparison free of statistical bias between schemes. For each SNR point, the modulated signal s is corrupted by AWGN whose variance is set from the measured signal power as $\sigma^2 = P_s/\text{SNR}$. OOK and PRK use real Gaussian noise; BFSK uses complex Gaussian noise with $\sigma^2/2$ per dimension. The detectors are minimum-distance: threshold 0.5 for OOK, threshold 0 for PRK, and the comparison $\text{Im}(r) > \text{Re}(r)$ for BFSK.

1.4 Simulation Results

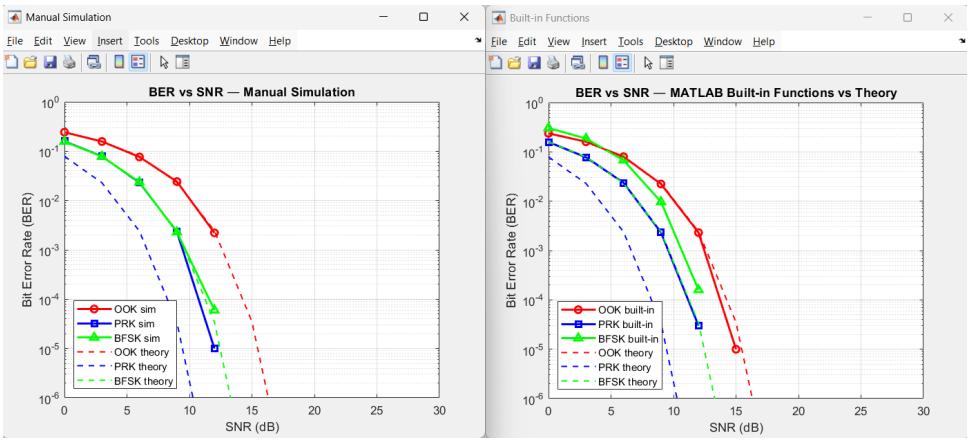

Figure 1: *Simulated BER vs. SNR for OOK, PRK, and BFSK.*

Three observations stand out from the simulated curves in Figure 1:

1. **PRK is the clear winner.** Its curve sits about 3 dB to the left of OOK and BFSK throughout the regime where errors are still measurable, matching the antipodal-vs-orthogonal prediction exactly. The PRK markers in the plot fall directly underneath the BFSK markers because both reach BER $\approx 10^{-5}$ near SNR = 12 dB, with PRK slightly lower as expected.

2. **OOK and BFSK track each other closely.** Although geometrically different (1-D unipolar vs. 2-D orthogonal), they share the same theoretical P_e expression, and the simulation confirms this: their curves are within statistical noise of one another, with OOK trailing BFSK by a fraction of a dB due to its less efficient power utilisation.

3. **The simulation floor at 10^{-6}.** Beyond SNR $\approx$ 15 dB the measured BER drops below the resolution afforded by 10^6 bits, so no points are plotted past that region. This is not the curves "converging" — the schemes continue to diverge by the same 3 dB indefinitely; the simulation simply cannot resolve errors that rare without more samples.

1.5 Conclusion (Part I)

The simulation reproduces the canonical ordering predicted by signal-space analysis: PRK $>$ BFSK $\approx$ OOK in BER performance, with PRK gaining a $\sim$3 dB edge that holds across the full operating range. PRK achieves this through antipodal signalling, which maximises Euclidean distance for a given energy; OOK pays a penalty for transmitting nothing half the time; BFSK uses an extra signalling dimension to obtain the same robustness as OOK without the on-off power asymmetry. In any energy-constrained link where coherent detection is feasible, PRK (BPSK) is the preferred choice — a conclusion strongly supported by both the theory and the simulated curves above.

2 Part II: Comparison of M-ary ASK ($M = 2, 4, 8$)

2.1 Simulation Results

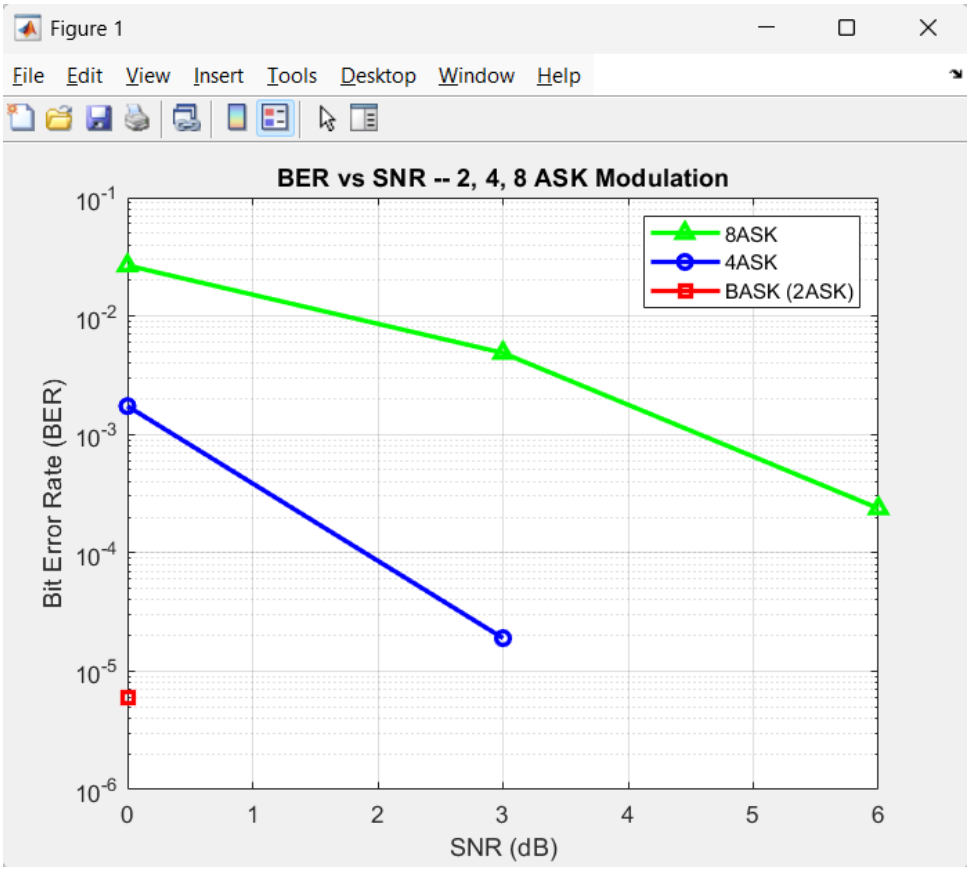

Figure 2: *Simulated BER vs. SNR for BASK (red), 4ASK (blue), and 8ASK (green).*

2.2 Discussion

As far as the bit error rate is concerned, the BASK (2-ASK) modulation scheme performs exceptionally well compared to its higher-order counterparts. From Figure 2, it is evident that its bit error rate is almost negligible at an SNR of 0 dB. As the SNR increases, the BASK plot no longer appears given that the bit error rate becomes too negligible. It falls below the comprehendible resolution of the plot (which is 10^{-6}), meaning that the bit error rate falls below 10^{-6} (i.e. zero errors were detected during the 10^6 bit transmission).

Following along with this logic, the 4-ASK modulation scheme comes second, and the 8-ASK modulation scheme comes third. The evident performance difference boils down to several factors:

- **Symbol Distance and Average Power:** For lower-order modulation schemes (such as BASK), the number of transmitted symbols is much lower. For BASK, there are only two states to distinguish between, keeping the distance between the two symbols maximized relative to the additive noise power. As the number of symbols increases (higher modulation order), maintaining the same distance between adjacent symbols requires much greater average transmission power.

- **Decision Boundaries:** If transmission power is normalized, increasing the constellation size forces the symbols closer together. This leaves the detection SNR at the receiver side much smaller, allowing minor noise variations to push the received symbol across a decision boundary.
- **Inner vs. Outer Symbols:** This error susceptibility is amplified in 8-ASK because every inner symbol has two nearest neighbors instead of one, essentially doubling the probability of crossing a threshold compared to the outer symbols.

Looking at the required SNR for reliable transmission:

- BASK is nearly error-free from 0 dB, and practically error-free at 3 dB.
- 4-ASK is almost error-free at 6 dB.
- 8-ASK is almost error-free at 9 dB.

Despite the fact that BASK is the best in terms of absolute reliability, this does not mean it is definitively the "best" modulation technique overall. While highly power-efficient, it consumes significantly more bandwidth to achieve a specific data rate compared to 4-ASK or 8-ASK. Because 8-ASK transmits three bits per symbol ($\log_2(8) = 3$), it sends data three times faster than BASK for the exact same symbol rate and bandwidth. Therefore, the choice of M is a fundamental engineering trade-off between power efficiency (BER performance) and spectral efficiency (data rate).

3 Part III: Comparison of Rectangular BPSK, QPSK, 4-QAM, and 4-ASK

3.1 Simulation Results

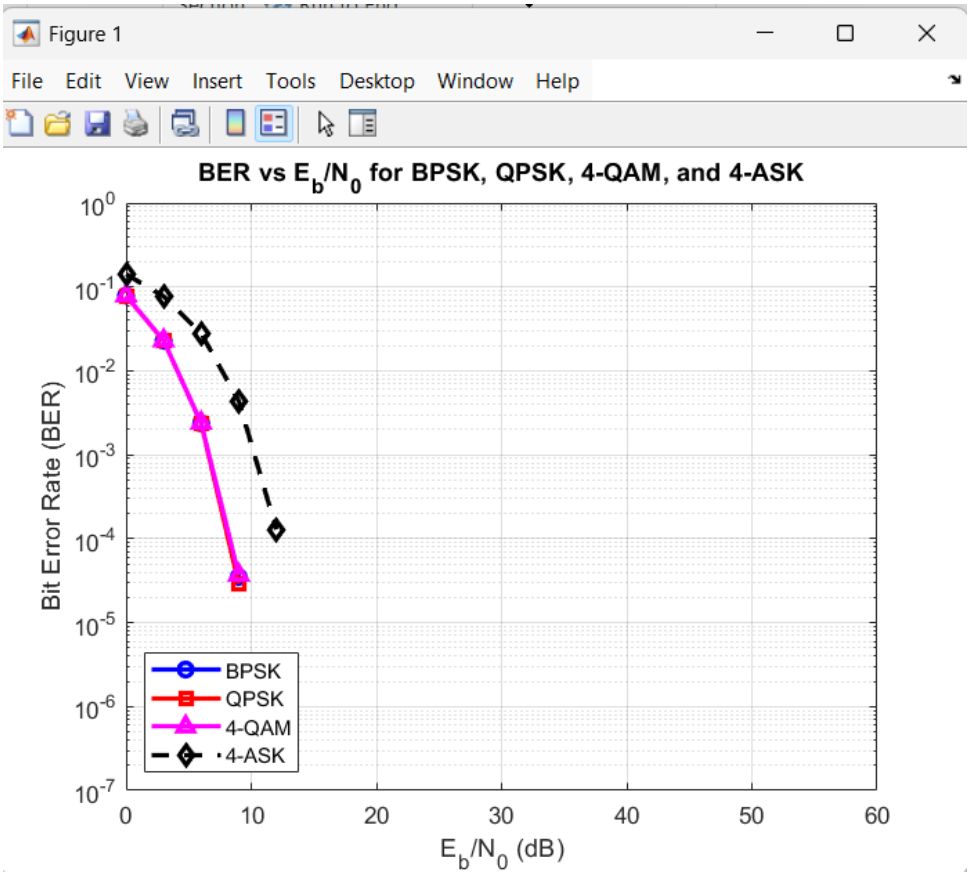

Figure 3: *Simulated BER vs. E_b/N_0 for BPSK, QPSK, 4-QAM, and 4-ASK.*

3.2 Discussion

Part III evaluates the performance of three fundamental phase/quadrature schemes alongside the 4-ASK scheme previously calculated in Part II, ensuring that all schemes are compared equitably on an energy-per-bit (E_b/N_0) basis.

The simulation reveals two distinct performance tiers:

1. **Overlapping Performance (BPSK, QPSK, 4-QAM):** As observed in the plot, the BER curves for BPSK, QPSK, and 4-QAM fall squarely on top of one another. This perfectly aligns with theoretical expectations. QPSK can be viewed mathematically as two orthogonal BPSK systems operating simultaneously (one on the In-Phase carrier, one on the Quadrature carrier). Because the noise in the orthogonal channels is independent, the bit error probability for each channel remains identical to that of a standard BPSK system. Furthermore, standard rectangular 4-QAM is geometrically and mathematically identical to QPSK, hence their indistinguishable error rates.

2. **The 4-ASK Penalty:** The 4-ASK curve is shifted to the right, indicating noticeably inferior BER performance compared to the phase/quadrature schemes. 4-ASK is a one-dimensional constellation. To pack four distinct symbols into a single dimension while maintaining the same average signal energy as the 2-D QPSK/4-QAM constellations, the physical distance between adjacent 4-ASK symbols must be compressed. This reduced Euclidean distance directly shrinks the decision regions, making the system far more vulnerable to AWGN.

Conclusion: For a system requiring 2 bits per symbol, QPSK (or 4-QAM) is definitively superior to 4-ASK. QPSK leverages a second orthogonal dimension to space out its symbols, maintaining the robust noise immunity of BPSK while doubling the spectral efficiency, whereas 4-ASK suffers an inherent power penalty by crowding all its symbols onto a single axis.

Appendix: Complete MATLAB Simulation Code

The following MATLAB script encompasses the logic, generation, modulation, AWGN addition, demodulation, and plotting for all three parts of the project.

```
% ================PART I=================
%% ========================================================
% Comparison of OOK, PRK, and BFSK Modulation Schemes
% Manual Simulation + MATLAB Built-in Functions
%% ========================================================
clear; clc; close all;
%% Parameters
N = 1e5;
SNR_dB = 0:3:30;
SNR_lin = 10.^(SNR_dB/10);
BER_OOK = zeros(size(SNR_dB));
BER_PRK = zeros(size(SNR_dB));
BER_BFSK = zeros(size(SNR_dB));
BER_OOK_builtin = zeros(size(SNR_dB));
BER_PRK_builtin = zeros(size(SNR_dB));
BER_BFSK_builtin = zeros(size(SNR_dB));
bits = randi([0 1],1,N);
%% Theoretical BER
BER_OOK_th = 0.5 * erfc(sqrt(SNR_lin/4));
BER_PRK_th = 0.5 * erfc(sqrt(SNR_lin));
BER_BFSK_th = 0.5 * erfc(sqrt(SNR_lin/2));
%% FSK Parameters
freqsep = 1;
nsamp = 8;
%% ========================================================
% SNR Loop
%% ========================================================
for k = 1:length(SNR_dB)
    SNR = SNR_lin(k);

    %% Manual OOK
    s_OOK = double(bits);
    Ps_OOK = mean(abs(s_OOK).^2);
    noise = sqrt(Ps_OOK/SNR) * randn(1,N);
    r_OOK = s_OOK + noise;
    det_OOK = r_OOK > 0.5;
    BER_OOK(k) = sum(det_OOK ~= bits)/N;

    %% Manual PRK
    s_PRK = 2*bits - 1;
    Ps_PRK = mean(abs(s_PRK).^2);
    noise = sqrt(Ps_PRK/SNR) * randn(1,N);
    r_PRK = s_PRK + noise;
    det_PRK = r_PRK > 0;
    BER_PRK(k) = sum(det_PRK ~= bits)/N;

    %% Manual BFSK
    s_BFSK = complex(double(bits==0), double(bits==1));
    Ps_BFSK = mean(abs(s_BFSK).^2);
    sigma = sqrt(Ps_BFSK/(2*SNR));
    noise = sigma * (randn(1,N) + 1i*randn(1,N));
    r_BFSK = s_BFSK + noise;
    det_BFSK = imag(r_BFSK) > real(r_BFSK);
    BER_BFSK(k) = sum(det_BFSK ~= bits)/N;

    %% Built-in OOK
    tx_OOK = pammod(bits,2);
    tx_OOK = (tx_OOK + 1)/2;
    Ps = mean(abs(tx_OOK).^2);
    noise = sqrt(Ps/SNR) * randn(1,N);
    rx_OOK = tx_OOK + noise;
    det_OOK_builtin = rx_OOK > 0.5;
    BER_OOK_builtin(k) = sum(det_OOK_builtin ~= bits)/N;

    %% Built-in PRK
```

```
    tx_PRK = pskmod(bits,2);
    Ps = mean(abs(tx_PRK).^2);
    noise = sqrt(Ps/SNR) * randn(1,N);
    rx_PRK = tx_PRK + noise;
    det_PRK_builtin = pskdemod(rx_PRK,2);
    BER_PRK_builtin(k) = sum(det_PRK_builtin ~= bits)/N;

    %% Built-in BFSK (Fixed)
    M = 2;
    tx_BFSK = fskmod(bits,M,freqsep,nsamp,nsamp);
    Ps = mean(abs(tx_BFSK).^2);
    SNR_sample = SNR/nsamp;
    sigma = sqrt(Ps / (2 * SNR_sample));
    noise = sigma * (randn(size(tx_BFSK)) + 1i*randn(size(tx_BFSK)));
    rx_BFSK = tx_BFSK + noise;
    det_BFSK_builtin = fskdemod(rx_BFSK,M,freqsep,nsamp,nsamp);
    BER_BFSK_builtin(k) = sum(det_BFSK_builtin ~= bits)/N;
end
%% ========================================================
% Figure 1     Manual Simulation
%% ========================================================
figure('Name','Manual Simulation','NumberTitle','off');
semilogy(SNR_dB,BER_OOK,'ro-','LineWidth',1.8,'MarkerSize',6);
hold on;
semilogy(SNR_dB,BER_PRK,'bs-','LineWidth',1.8,'MarkerSize',6);
semilogy(SNR_dB,BER_BFSK,'g^-','LineWidth',1.8,'MarkerSize',6);
semilogy(SNR_dB,BER_OOK_th,'r--','LineWidth',1.2);
semilogy(SNR_dB,BER_PRK_th,'b--','LineWidth',1.2);
semilogy(SNR_dB,BER_BFSK_th,'g--','LineWidth',1.2);
grid on;
xlabel('SNR (dB)');
ylabel('Bit Error Rate (BER)');
title('BER vs SNR     Manual Simulation');
legend('OOK sim','PRK sim','BFSK sim','OOK theory','PRK theory','BFSK theory','
    Location','southwest');
ylim([1e-6 1]);
xlim([0 30]);

% ================PART II=================
% Constants
fs = 20000;
fc = 1000;
Rb = 1000;
Tb = 1/Rb;
% Input random bits --> 10^6 bits
bits = 1e6;
% Vector of random bits
vector = randi([0,1],1,bits);
% SNR in dB, incrementing by 3 dB from 0 to 60 dB
snr = 0:3:60;
% BASK
% Initialize BER vector to store results for each SNR
BER_BASK = zeros(1,length(snr));
% Samples per bit to represent each symbol
samples_per_bit = fs/Rb;
% Define orthonormal basis for ASK
t_bit = 0:1/fs:Tb-1/fs;
% Carrier Signal Generation
E_bit = Tb / 2;
A = sqrt(E_bit)/2;
% Symbol mask 0 --> -A and 1 --> A
symbol_mask = (2*vector - 1) * A;
phi_I_BASK = sqrt(2/Tb) * cos(2*pi*fc*t_bit);
% Transmission:
% Multiply the basis by the transmission vector
y_matrix_form = phi_I_BASK' .* symbol_mask;
% Reshape the transmission into a continuous frame
y = reshape(y_matrix_form, 1, []);
decision_boundary = 0;
for i = 1:length(snr)
    % Add noise to the input vector
    Rx_sequence = awgn(y, snr(i), 'measured');
    % Demodulate the transmitted waveform
```

```
    Rx_matrix = reshape(Rx_sequence, samples_per_bit, bits);
    % Multiply by the basis function and integrate
    I_received = sum(Rx_matrix .* phi_I_BASK') * (1/fs);
    noisy_vector = I_received >= decision_boundary;
    % Calculate the Bit Error Rate (BER) using the XOR operator
    errors = sum(xor(vector, noisy_vector));
    BER_BASK(i) = errors / bits;
end

% 4-ASK
BER_4ASK = zeros(1,length(snr));
Ts = 2 * Tb;
samples_per_symbol = Ts * fs;
bits_grouped = reshape(vector, 2, bits/2);
E_bit = Tb / 2;
A = sqrt(E_bit)/2;
% Gray coding
symbol_index = bits_grouped(1,:)*2 + bits_grouped(2,:);
symbol_mask = zeros(1,bits/2);
symbol_mask(symbol_index == 0) = -3 * A;
symbol_mask(symbol_index == 1) = -1 * A;
symbol_mask(symbol_index == 3) = 1 * A;
symbol_mask(symbol_index == 2) = 3 * A;
t_sym = 0:1/fs:Ts-1/fs;
phi_I_4ASK = sqrt(2/Ts) * cos(2*pi*fc*t_sym);
y_matrix_form = phi_I_4ASK' .* symbol_mask;
y = reshape(y_matrix_form, 1, []);
for i = 1:length(snr)
    Rx_sequence = awgn(y, snr(i), 'measured');
    Rx_matrix = reshape(Rx_sequence, samples_per_symbol, bits/2);
    I_received = sum(Rx_matrix .* phi_I_4ASK') * (1/fs);
    rx_bits = zeros(2,bits/2);
    idx = I_received > 2*A;
    rx_bits(1,idx) = 1; rx_bits(2,idx) = 0;
    idx = (I_received > 0) & (I_received <= 2*A);
    rx_bits(1,idx) = 1; rx_bits(2,idx) = 1;
    idx = (I_received <= 0) & (I_received > -2*A);
    rx_bits(1,idx) = 0; rx_bits(2,idx) = 1;
    idx = I_received <= -2*A;
    rx_bits(1,idx) = 0; rx_bits(2,idx) = 0;
    rx_vector = reshape(rx_bits, 1, []);
    errors = sum(xor(vector, rx_vector));
    BER_4ASK(i) = errors / bits;
end

% 8-ASK
BER_8ASK = zeros(1,length(snr));
bits = 999996;
vector = randi([0,1],1,bits);
Ts = 3 * Tb;
samples_per_symbol = Ts * fs;
bits_grouped = reshape(vector, 3, bits/3);
E_bit = Tb / 2;
A = sqrt(E_bit)/2;
symbol_index = bits_grouped(1,:)*4 + bits_grouped(2,:)*2 + bits_grouped(3,:);
symbol_mask = zeros(1,bits/3);
symbol_mask(symbol_index == 0) = -7 * A;
symbol_mask(symbol_index == 1) = -5 * A;
symbol_mask(symbol_index == 3) = -3 * A;
symbol_mask(symbol_index == 2) = -1 * A;
symbol_mask(symbol_index == 6) = 1 * A;
symbol_mask(symbol_index == 7) = 3 * A;
symbol_mask(symbol_index == 5) = 5 * A;
symbol_mask(symbol_index == 4) = 7 * A;
t_sym = 0:1/fs:Ts-1/fs;
phi_I_8ASK = sqrt(2/Ts) * cos(2*pi*fc*t_sym);
y_matrix_form = phi_I_8ASK' .* symbol_mask;
y = reshape(y_matrix_form, 1, []);
for i = 1:length(snr)
    Rx_sequence = awgn(y, snr(i), 'measured');
    Rx_matrix = reshape(Rx_sequence, samples_per_symbol, bits/3);
    I_received = sum(Rx_matrix .* phi_I_8ASK') * (1/fs);
    rx_bits = zeros(3,bits/3);
```

```
    idx = I_received > 6*A;
    rx_bits(1,idx) = 1; rx_bits(2,idx) = 0; rx_bits(3,idx) = 0;
    idx = (I_received > 4*A) & (I_received <= 6*A);
    rx_bits(1,idx) = 1; rx_bits(2,idx) = 0; rx_bits(3,idx) = 1;
    idx = (I_received > 2*A) & (I_received <= 4*A);
    rx_bits(1,idx) = 1; rx_bits(2,idx) = 1; rx_bits(3,idx) = 1;
    idx = (I_received > 0) & (I_received <= 2*A);
    rx_bits(1,idx) = 1; rx_bits(2,idx) = 1; rx_bits(3,idx) = 0;
    idx = (I_received > -2*A) & (I_received <= 0);
    rx_bits(1,idx) = 0; rx_bits(2,idx) = 1; rx_bits(3,idx) = 0;
    idx = (I_received > -4*A) & (I_received <= -2*A);
    rx_bits(1,idx) = 0; rx_bits(2,idx) = 1; rx_bits(3,idx) = 1;
    idx = (I_received > -6*A) & (I_received <= -4*A);
    rx_bits(1,idx) = 0; rx_bits(2,idx) = 0; rx_bits(3,idx) = 1;
    idx = (I_received <= -6*A);
    rx_bits(1,idx) = 0; rx_bits(2,idx) = 0; rx_bits(3,idx) = 0;
    rx_vector = reshape(rx_bits, 1, []);
    errors = sum(xor(vector, rx_vector));
    BER_8ASK(i) = errors / bits;
end

% Plot BER vs SNR Part II
figure;
semilogy(snr, BER_8ASK, 'g-^', 'LineWidth', 2);
hold on;
semilogy(snr, BER_4ASK, 'b-o', 'LineWidth', 2);
hold on;
semilogy(snr, BER_BASK, 'r-s', 'LineWidth', 2);
grid on;
xlabel('SNR (dB)');
ylabel('Bit Error Rate (BER)');
title('BER vs SNR -- 2, 4, 8 ASK Modulation');
legend('8ASK', '4ASK', 'BASK (2ASK)');

% ================ PART III =================
%% Compare the Performance of BPSK, QPSK, 4-QAM, and 4-ASK
clear; clc; close all;
%% ---------- Simulation Parameters ----------
N   = 1e6;          % number of bits
snr = 0:3:60;       % Eb/N0 range in dB
Rb  = 1000;         % bit rate
Tb  = 1/Rb;         % bit period
Eb = Tb/2;
A = sqrt(Eb);

%% ==========================================================
%% BPSK
fprintf('Simulating BPSK...\n');
bits_BPSK = randi([0 1], 1, N);
tx_BPSK = (2*bits_BPSK - 1) * A;
BER_BPSK = zeros(size(snr));
for i = 1:length(snr)
    SNR_lin = 10^(snr(i)/10);
    sigma = sqrt(Eb/(2*SNR_lin));
    rx = tx_BPSK + sigma*randn(1,N);
    rx_bits = rx >= 0;
    BER_BPSK(i) = sum(bits_BPSK ~= rx_bits)/N;
end

%% ==========================================================
%% QPSK (Gray coded)
fprintf('Simulating QPSK...\n');
bits_QPSK = randi([0 1], 1, N);
num_sym_QPSK = N/2;
bits_pairs = reshape(bits_QPSK,2,num_sym_QPSK);
I_QPSK =  (2*bits_pairs(1,:) - 1) * A;
Q_QPSK = -(2*bits_pairs(2,:) - 1) * A;
BER_QPSK = zeros(size(snr));
for i = 1:length(snr)
    SNR_lin = 10^(snr(i)/10);
    sigma = sqrt(Eb/(2*SNR_lin));
    I_rx = I_QPSK + sigma*randn(1,num_sym_QPSK);
    Q_rx = Q_QPSK + sigma*randn(1,num_sym_QPSK);
```

```
    rx_b1 = I_rx >= 0;
    rx_b2 = Q_rx <= 0;
    rx_bits = reshape([rx_b1; rx_b2],1,[]);
    BER_QPSK(i) = sum(bits_QPSK ~= rx_bits)/N;
end

%% ==========================================================
%% 4-QAM
fprintf('Simulating 4-QAM...\n');
bits_4QAM = randi([0 1],1,N);
num_sym_4QAM = N/2;
bits_pairs4 = reshape(bits_4QAM,2,num_sym_4QAM);
I_4QAM = (2*bits_pairs4(1,:) - 1) * A;
Q_4QAM = (2*bits_pairs4(2,:) - 1) * A;
BER_4QAM = zeros(size(snr));
for i = 1:length(snr)
    SNR_lin = 10^(snr(i)/10);
    sigma = sqrt(Eb/(2*SNR_lin));
    I_rx4 = I_4QAM + sigma*randn(1,num_sym_4QAM);
    Q_rx4 = Q_4QAM + sigma*randn(1,num_sym_4QAM);
    rx_b1 = I_rx4 >= 0;
    rx_b2 = Q_rx4 >= 0;
    rx_bits4 = reshape([rx_b1; rx_b2],1,[]);
    BER_4QAM(i) = sum(bits_4QAM ~= rx_bits4)/N;
end

%% ==========================================================
%% 4-ASK (Gray coded)
fprintf('Simulating 4-ASK...\n');
bits_4ASK = randi([0 1],1,N);
num_sym_4ASK = N/2;
bits_pairs_4ASK = reshape(bits_4ASK,2,num_sym_4ASK);
sym_idx = bits_pairs_4ASK(1,:)*2 + bits_pairs_4ASK(2,:);
tx_4ASK = zeros(1,num_sym_4ASK);
tx_4ASK(sym_idx == 0) = -3*A;   % 00
tx_4ASK(sym_idx == 1) = -1*A;   % 01
tx_4ASK(sym_idx == 3) = +1*A;   % 11
tx_4ASK(sym_idx == 2) = +3*A;   % 10
Es_4ASK = mean(tx_4ASK.^2);
Eb_4ASK = Es_4ASK / 2;
BER_4ASK = zeros(size(snr));
for i = 1:length(snr)
    SNR_lin = 10^(snr(i)/10);
    sigma = sqrt(Eb_4ASK/(2*SNR_lin));
    rx_4ASK = tx_4ASK + sigma*randn(1,num_sym_4ASK);
    rx_bits_4ASK = zeros(2,num_sym_4ASK);
    idx = rx_4ASK > 2*A;
    rx_bits_4ASK(:,idx) = repmat([1;0],1,sum(idx));
    idx = (rx_4ASK > 0) & (rx_4ASK <= 2*A);
    rx_bits_4ASK(:,idx) = repmat([1;1],1,sum(idx));
    idx = (rx_4ASK <= 0) & (rx_4ASK > -2*A);
    rx_bits_4ASK(:,idx) = repmat([0;1],1,sum(idx));
    idx = rx_4ASK <= -2*A;
    rx_bits_4ASK(:,idx) = repmat([0;0],1,sum(idx));
    rx_bits4ASK = reshape(rx_bits_4ASK,1,[]);
    BER_4ASK(i) = sum(bits_4ASK ~= rx_bits4ASK)/N;
end

%% ==========================================================
%% Plot Results
figure('Color','w');
semilogy(snr, BER_BPSK, 'b-o', 'LineWidth',2,'MarkerSize',6);
hold on;
semilogy(snr, BER_QPSK, 'r-s', 'LineWidth',2,'MarkerSize',6);
semilogy(snr, BER_4QAM, 'm-^', 'LineWidth',2,'MarkerSize',6);
semilogy(snr, BER_4ASK, 'k--d', 'LineWidth',2,'MarkerSize',6);
grid on;
xlabel('E_b/N_0 (dB)');
ylabel('Bit Error Rate (BER)');
title('BER vs E_b/N_0 for BPSK, QPSK, 4-QAM, and 4-ASK');
legend('BPSK', 'QPSK', '4-QAM', '4-ASK', 'Location','southwest');
xlim([snr(1) snr(end)]);
ylim([1e-7 1]);
```